

APOSTILA DE COMPILADORES

Aula 12

Integração entre scanner, parser e AST

Nós de declaração • expressões • atribuições • comandos de controle • chamadas de funções • tratamento de erros • impressão textual da AST • testes de integração

Ciência da Computação — 2026/2

Professor: Alex Torquato Souza Carneiro

Projeto integrador: linguagem MINIC

Material de apoio baseado no Capítulo 4, Seções 4.7 a 4.9, e nas Seções 2.4 e 2.5 de Aho, Sethi e Ullman.

Apresentação

Esta apostila acompanha a Aula 12 e trata da passagem entre as etapas de reconhecimento sintático e construção de uma representação útil para o restante do compilador. O foco não é apenas aceitar ou rejeitar uma sequência de tokens: o parser deve produzir uma árvore sintática abstrata coerente, preservar posições no código e emitir diagnósticos que ajudem o programador a corrigir seus erros.

A aula conecta os conceitos das aulas anteriores — gramáticas livres de contexto, análise descendente, LL(1), análise ascendente e famílias LR — a uma implementação concreta do front-end da MINIC. O scanner fornece tokens; o parser organiza esses tokens segundo a gramática; as ações semânticas constroem nós da AST; e os testes verificam o comportamento integrado.

Objetivos de aprendizagem

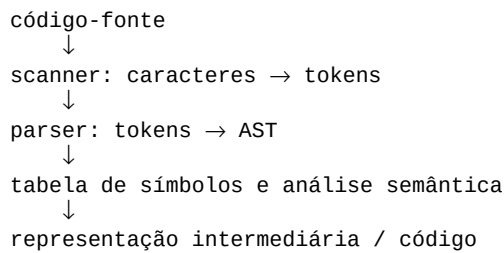
- integrar a interface do scanner com o parser por meio de tokens consistentes;
- definir nós para declarações, expressões, comandos, funções e chamadas;
- construir ASTs durante a análise sintática sem confundir árvore concreta com árvore abstrata;
- projetar mensagens de erro com token, lexema, linha, coluna e contexto esperado;
- imprimir a AST em formato textual estável e legível;
- planejar testes de unidade e testes de integração para entradas válidas e inválidas.

Relação com a sequência do curso

A Aula 11 apresentou o parsing bottom-up, as reduções e os estados LR. Nesta aula, independentemente de o parser ser descendente ou ascendente, as decisões de reconhecimento passam a produzir estruturas de dados. A diferença está na estratégia de análise; o contrato de saída — uma AST ou uma coleção estruturada de erros — permanece o mesmo.

1. O front-end integrado

O front-end de um compilador transforma o texto-fonte em uma representação intermediária organizada. Para a MINIC, podemos visualizar o fluxo da seguinte maneira:



O scanner não deve decidir se uma atribuição está dentro de um bloco ou se uma chamada possui a quantidade correta de argumentos. Essas são relações sintáticas ou semânticas. Ele deve, contudo, entregar tokens confiáveis: tipo, lexema e posição. O parser consulta principalmente o tipo para escolher produções e utiliza o lexema para construir identificadores, literais e diagnósticos.

Campo	Função no parser
tipo	orienta a escolha da produção e o reconhecimento de terminais.
lexema	preserva o texto original de identificadores, literais e operadores.
linha/coluna	permite localizar o erro e anotar a origem de um nó da AST.
fim de entrada	marca o encerramento da sequência e permite a aceitação.

A interface deve ser definida antes da implementação. Um parser que espera `TOKEN_ID`, mas recebe apenas strings, ou que recebe posições em formato inconsistente, pode produzir erros difíceis de diagnosticar mesmo quando a gramática está correta.

2. Da análise sintática à AST

A análise sintática determina como a cadeia de tokens pode ser gerada pela gramática. A AST, por sua vez, registra somente as construções semanticamente relevantes. Em uma expressão, os nós interiores representam operadores e os filhos representam operandos. Em um comando, a raiz pode representar a própria construção, como If, While, Return ou Assign.

A árvore concreta explicita todos os símbolos usados na derivação, inclusive não terminais auxiliares e delimitadores. A AST elimina esses detalhes quando eles não são necessários para a análise semântica ou para a geração de código. Parênteses, por exemplo, podem desaparecer quando sua função já estiver refletida na hierarquia dos nós.

cap02c_AHO.qxd 26.10.07 12:39 Page 44

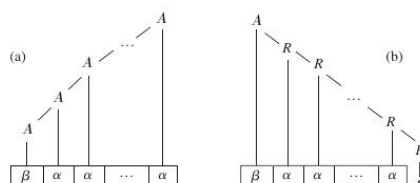


FIGURA 2.20 Formas recursivas à esquerda e à direita para gerar uma sequência de símbolos.

Na Seção 4.3.3, vamos considerar formas mais gerais de recursão à esquerda e mostrar como toda a recursão à esquerda pode ser eliminada de uma gramática.

2.4.6 EXERCÍCIOS DA SEÇÃO 2.4

Exercício 2.4.1: Construa analisadores de descida recursiva, começando com as seguintes gramáticas:

- $S \rightarrow + S S \mid - S S \mid a$
- $S \rightarrow S (S) S \mid \epsilon$
- $S \rightarrow 0 S 1 \mid 0 1$

2.5 UM TRADUTOR PARA EXPRESSÕES SIMPLES

Usando as técnicas das três últimas seções, vamos construir agora um tradutor dirigido por sintaxe, na forma de um programa Java completo, que traduz expressões aritméticas para a forma pós-fixada. Para manter o programa inicial pequeno, começamos com expressões consistindo em dígitos separados pelos operadores binários de adição e de subtração. Estendemos o programa na Seção 2.6 para traduzir expressões que incluem números e outros operadores. Vale a pena estudar a tradução de expressões em detalhes, pois elas aparecem como uma construção em muitas linguagens.

Um esquema de tradução dirigido por sintaxe normalmente serve como a especificação para um tradutor. O esquema na Figura 2.21 (repetido da Figura 2.15) define a tradução a ser realizada nesta seção.

$expr$	$\rightarrow$	$expr_1 + term$	{print(' + ')}
		$expr_1 - term$	{print(' - ')}
		$term$	
$term$	$\rightarrow$	0	{print(' 0 ')}
		1	{print(' 1 ')}
		...	
		9	{print(' 9 ')}

FIGURA 2.21 Ações de tradução para notação pós-fixada.

Normalmente, a gramática básica de determinado esquema precisa ser modificada antes que possa ser reconhecida com um analisador preditivo. Em particular, a gramática na Figura 2.21 é recursiva à esquerda e, como vimos na seção anterior, um analisador preditivo não pode tratar uma gramática recursiva à esquerda.

Parece que temos um conflito: por um lado, precisamos de uma gramática que facilite a tradução e, por outro, precisamos de uma gramática significativamente diferente que facilite a análise sintática. A solução é começar com a gramática para tradução fácil e transformá-la cuidadosamente para facilitar a análise. Eliminando a recursão à esquerda na Figura 2.21, podemos obter uma gramática adequada para uso em um tradutor de descida recursiva preditivo.

2.5.1 SINTAXE ABSTRATA E CONCRETA

Um ponto de partida útil para projetar um tradutor é uma estrutura de dados chamada árvore sintática abstrata. Em uma *árvore sintática abstrata*, para uma expressão, cada nó interior representa um operador; os filhos do nó representam os operandos do operador. De modo geral, qualquer construção de linguagem de programação pode ser tratada criando um operador para a construção e tratando como operandos os componentes semanticamente significativos dessa construção.

Figura 1 — Trecho do livro sobre sintaxe abstrata e concreta. Fonte: Aho, Sethi e Ullman, Cap. 2, material fornecido para a disciplina.

Critério prático: se uma informação será necessária para verificar tipos, resolver nomes, avaliar a expressão ou gerar código, ela deve ser preservada na AST ou em atributos associados. Se serve apenas para orientar a derivação, pode ser omitida.

3. Contrato dos nós da AST

Uma AST deve possuir uma estrutura regular. Todos os nós podem compartilhar informações de origem e, depois, especializar seus campos conforme a construção representada. Uma definição didática é:

Node

```

■■■■ kind      # categoria do nó
■■■■ line, col  # posição inicial
■■■■ children   # subárvores
  
```

```

Identifier(name, line, col)
Literal(value, type, line, col)
Binary(op, left, right, line, col)
Assign(target, value, line, col)
Block(declarations, statements, line, col)
  
```

A posição no nó não é mero detalhe de depuração. Ela será usada para mensagens da análise semântica, como “variável não declarada na linha 8, coluna 5”, e para apontar a origem de uma expressão incompatível. Também é útil preservar o lexema original quando a representação interna converte o valor, por exemplo, de texto para inteiro.

Categoria	Campos essenciais	Exemplo
Declaração	tipo, nome, inicializador opcional	<code>int x;</code>
Expressão	operador e operandos	<code>a + b * c</code>
Comando	condição, corpo, alternativa	<code>if (x) y = 1;</code>
Função	nome, parâmetros, retorno, corpo	<code>int f(int x) { return x; }</code>

4. Nós de declaração e organização do programa

Na MINIC, o programa pode ser entendido como uma lista de declarações globais. Uma declaração pode introduzir uma variável ou uma função. A AST deve preservar a ordem em que essas construções aparecem, pois a ordem pode ser relevante para diagnósticos, escopo e geração de código.

```
Program
  ■■■ declarations
    ■■■ VarDecl(type=int, name=x)
    ■■■ FunctionDecl(name=main, return=int)
      ■■■ params
      ■■■ body: Block(...)
```

Durante uma análise descendente, o nó pode ser criado quando a produção é reconhecida. Em um parser ascendente, ele pode ser criado no momento da redução correspondente. O princípio é o mesmo: os filhos devem estar disponíveis quando o pai for construído. Uma redução de “tipo identificador ponto e vírgula” pode produzir diretamente um nó VarDecl.

Declaração de variável

var-declaration → type ID ;

Ao reconhecer: int total ;
produzir: VarDecl(type="int", name="total")

Declarações, em geral, não precisam manter na AST todos os tokens delimitadores. O ponto e vírgula comprova o encerramento sintático, mas não é uma operação que as fases posteriores precisem visitar. O tipo, o nome, o inicializador e a posição, entretanto, devem ser preservados.

5. Expressões e atribuições

Expressões são uma das partes mais frequentes da AST. A gramática organiza precedência e associatividade; as ações do parser convertem essa organização em nós operadores. Para a expressão abaixo:

```
x = 2 + 5 * 2;
```

a raiz é uma atribuição. O lado direito é uma adição cujo segundo operando é uma multiplicação. Os nós auxiliares da gramática — como E, T e F — não aparecem necessariamente na árvore final.

```
Assign
  Identifier("x")
  Binary("+")
    Literal(2)
    Binary("*")
      Literal(5)
      Literal(2)
```

Uma implementação deve distinguir o alvo de uma atribuição de uma expressão qualquer. Em MINIC, o lado esquerdo pode ser um identificador ou, se a linguagem permitir, um acesso a vetor. O parser verifica a forma sintática; a análise semântica verificará se o alvo é atribuível e se os tipos são compatíveis.

Ações típicas

- ao reconhecer um literal, criar `Literal` com valor convertido e posição;
- ao reconhecer um identificador, criar `Identifier` com lexema e posição;
- ao reduzir uma operação binária, criar `Binary` e ligar esquerda e direita;
- ao reconhecer “alvo = expressão”, criar `Assign`;
- ao reconhecer parênteses, retornar a subexpressão, sem criar um nó obrigatório para o delimitador.

6. A integração com a descida recursiva e com LR

Em um parser descendente, cada procedimento pode retornar um nó. Por exemplo, `parseExpression` devolve a raiz da expressão reconhecida. Em um parser shift-reduce, a pilha de valores semânticos acompanha a pilha de estados: um shift empilha o valor do token; uma redução remove os filhos e empilha o novo nó.

```
shift ID("x")      → empilha Identifier("x")
shift INT(2)       → empilha Literal(2)
reduce F → id      → mantém o nó do identificador
reduce T → T * F   → cria Binary("*", esquerdo, direito)
```

A ideia de associar ações à gramática é central nos geradores de parser. Aho, Sethi e Ullman apresentam a construção de tradutores dirigidos por sintaxe e o uso de geradores LALR. O parser gerado executa ações quando reconhece reduções, permitindo construir a AST sem percorrer novamente toda a entrada.

cap04_AHOg.qxd 26.10.07 14:34 Page 183



CAPÍTULO 4: ANÁLISE SINTÁTICA

183

e4: Chamado do estado 6 quando o fim da entrada é encontrado.

empilhe o estado 9 (para um parêntese direito);
emite diagnóstico "faltando um parêntese direito".

Na entrada errônea `id + )`, a sequência de configurações entradas pelo analisador sintático aparece na Figura 4.54.

PILHA	SÍMBOLOS	ENTRADA	AÇÃO
0		id +) \$	
0 3	id	+) \$	
0 1	<i>E</i>	+) \$	
0 1 4	<i>E</i> +	) \$	"parêntese direito não balanceado";
0 1 4	<i>E</i> +	\$	e2 remove parêntese direito
0 1 4 3	<i>E</i> + id	\$	"faltando operando";
0 1 4 7	<i>E</i> +	\$	e 1 empilha estado 3
0 1	<i>E</i> +	\$	

FIGURA 4.54 Movimentos de análise e recuperação de erros feitos por um analisador sintático LR.

4.8.4 EXERCÍCIOS DA SEÇÃO 4.8

Exercício 4.8.1: A seguir, temos uma gramática ambígua para as expressões com n operadores binários, infixos, em n diferentes níveis de precedência:

$$E \rightarrow E \theta_1 E \mid E \theta_2 E \mid \dots \mid E \theta_n E \mid (E) \mid \text{id}$$

- Como uma função de n , quais são os conjuntos de itens SLR?
- Como você resolveria os conflitos nos itens SLR de modo que todos os operadores sejam associativos à esquerda, e θ_1 tenha precedência sob θ_2 , que tem precedência sob θ_3 , e assim por diante?
- Mostre a tabela de análise SLR que resulta das suas decisões na parte (b).
- Repita as partes (a) e (c) para a gramática não-ambígua, que define o mesmo conjunto de expressões, mostrado na Figura 4.55.
- Como as contagens do número de conjuntos de itens e os tamanhos das tabelas para as duas gramáticas (ambígua e não-ambígua) se comparam? O que essa comparação lhe diz sobre o uso das gramáticas de expressão ambígua?

$$\begin{aligned} E_1 &\rightarrow E_1 \theta E_2 \mid E_2 \\ E_2 &\rightarrow E_2 \theta E_3 \mid E_3 \\ &\vdots \\ E_n &\rightarrow E_n \theta E_{n+1} \mid E_{n+1} \\ E_{n+1} &\rightarrow (E_1) \mid \text{id} \end{aligned}$$

FIGURA 4.55 Gramática não-ambígua para n operadores.

! Exercício 4.8.2: Na Figura 4.56, temos uma gramática para certas construções, semelhantemente ao que discutimos no Exercício 4.4.12. Mais uma vez, e e s são terminais representando expressões condicionais e "outros comandos", respectivamente.

- Construa uma tabela de análise LR para essa gramática, resolvendo os conflitos de forma usual para o problema do else vazio.
- Implemente a correção de erro preenchendo as entradas em branco da tabela de análise com ações de redução extras ou rotinas adequadas de recuperação de erros.
- Mostre o comportamento do seu analisador sintático para as seguintes entradas:
 - `if e then s ; if s then s end`
 - `while e do begin s ; if e then s ; end`

4.9 GERADORES DE ANALISADORES SINTÁTICOS

Esta seção mostra como um gerador de analisador sintático pode ser usado para facilitar a construção do *front-end* de um compilador. Usaremos o gerador de analisadores LALR `Yacc` como base de nossa discussão, pois ele implementa muitos dos conceitos ilustrados nas duas seções anteriores e está disponível para uso. `Yacc` significa "yet another compiler-compiler" (ainda

Figura 2 — Trecho do livro sobre geradores de analisadores sintáticos. Fonte: Aho, Sethi e Ullman, Cap. 4.

A escolha entre parser descendente e ascendente não altera o contrato da AST. Altera a forma como as produções são reconhecidas e o momento em que cada ação é executada.

7. Comandos de controle

Comandos de controle representam fluxo de execução. A AST deve tornar explícita a condição e os corpos envolvidos, porque fases posteriores precisarão gerar desvios, rótulos ou instruções de interpretação.

```
if (x < 10)
    x = x + 1;
else
    x = 0;

If
  Binary("<")
  Identifier("x")
  Literal(10)
  Assign("x", Binary("+", Identifier("x"), Literal(1)))
  Assign("x", Literal(0))
```

O parser deve resolver a estrutura sintática do if e associar corretamente o else. A ambiguidade clássica do “else pendente” precisa ser tratada pela gramática, por uma convenção — normalmente associando o else ao if mais próximo — ou por uma regra explícita do gerador de parser.

Para while e outros laços, o nó deve conter a condição e o corpo. Comandos como break e continue podem ser nós próprios, pois terão comportamento específico na geração de código e precisam ser validados em relação ao contexto de um laço.

Exemplo de nós

Comando	Estrutura da AST
If	condição, então, senão opcional
While	condição, corpo
Return	expressão opcional
Block	lista ordenada de declarações e comandos

8. Funções, parâmetros e chamadas

Funções introduzem uma relação entre declaração, parâmetros, corpo e chamadas. O parser deve construir a forma sintática; a tabela de símbolos e a análise semântica verificarão se o nome existe, se o número de argumentos coincide e se os tipos são compatíveis.

```
FunctionDecl
  name: soma
  returnType: int
  params
    Param(int, a)
    Param(int, b)
  body: Block(...)

Call
  callee: Identifier("soma")
  args
    Identifier("x")
    Literal(2)
```

Uma chamada deve preservar a ordem dos argumentos. Essa ordem é essencial tanto para a verificação semântica quanto para a convenção de chamada na geração de código. Uma lista vazia de argumentos deve ser representada de maneira válida, e não confundida com ausência de uma chamada.

Durante uma redução de chamada, o parser reúne o identificador da função e a lista de expressões já construídas. Em uma estratégia descendente, uma função `parseArgumentList` pode acumular os elementos até encontrar o parêntese direito. Em ambos os casos, vírgulas e parênteses são delimitadores sintáticos, enquanto os argumentos tornam-se filhos da AST.

Exemplo MINIC

```
int soma(int a, int b) {
    return a + b;
}

int resultado;
resultado = soma(3, 4);
```

9. Tratamento de erros sintáticos

Um parser útil não deve apenas falhar; deve explicar o que encontrou, onde encontrou e o que esperava. Uma mensagem mínima contém o tipo do erro, o token inesperado, o lexema, a linha, a coluna e um conjunto de expectativas.

Situação	Mensagem orientadora
Operando ausente	“esperado identificador, literal ou parêntese após o operador +”
Parêntese não fechado	“esperado) antes do fim do comando, linha 8”
Ponto e vírgula ausente	“esperado ; após a declaração de variável”
Token inesperado	“encontrado else; esperado comando ou }”

Em parser preditivo, a recuperação pode usar modo pânico: descartar tokens até encontrar um sincronizador, como ponto e vírgula, chave direita, parêntese direito ou palavra-chave que inicie uma construção. Em parser LR, a recuperação pode remover estados e símbolos até alcançar um estado que aceite um token de retomada.

A recuperação precisa evitar dois extremos: interromper no primeiro erro, perdendo diagnósticos úteis, ou descartar tokens demais e produzir mensagens em cascata. Cada erro registrado deve preservar a posição original e o parser deve marcar claramente quando entrou em modo de recuperação.

- Dê o diagrama de transição (NFA) para os itens válidos dessa gramática de acordo com a regra dada na caixa citada anteriormente.
- Aplique a construção de subconjuntos (Algoritmo 3.20) ao seu NFA da parte (a). Compare o DFA resultante com o conjunto de itens LR(0) para a gramática.
- Mostre que, em todos os casos, a construção de subconjuntos aplicada ao NFA que vem dos itens válidos para uma gramática produz os conjuntos de itens LR(0).

! **Exercício 4.6.9:** Dada a gramática ambígua:

$$\begin{aligned} S &\rightarrow AS|b \\ A &\rightarrow SA|a \end{aligned}$$

Construa, para essa gramática, sua coleção de conjuntos de itens LR(0). A tabela LR construída para essa gramática possui entradas com conflito. Quais são elas? Suponha que tentássemos usar essa tabela LR escolhendo, de forma não-determinista, uma das ações possíveis sempre que houvesse um conflito. Mostre todas as seqüências de ações possíveis para a entrada *abab*.

4.7 ANALISADORES SINTÁTICOS LR MAIS PODEROSOS

Nesta seção, estenderemos as técnicas de análise LR anteriores incorporando nos itens o primeiro símbolo da entrada, ainda não lido. Denominamos este símbolo de lookahead. Existem dois métodos LR com esta característica:

- O método “*LR canônico*”, ou apenas “*LR*”, faz uso do(s) símbolo(s) lookahead. Esse método usa uma tabela construída a partir do conjunto de itens denominados itens LR(1).
- O método “*Look Ahead LR*”, ou “*LALR*”, cuja tabela é construída a partir dos conjuntos de itens LR(0), e possui muito menos estados que os analisadores sintáticos típicos, baseados no conjunto canônico de itens LR(1). A incorporação de símbolos lookaheads nos itens LR(0) introduz muito mais poder ao método e o torna mais geral, permitindo-lhe tratar muito mais gramáticas que o método SLR. Além de o LALR ser mais poderoso que o SLR, suas tabelas ACTION e GOTO não são maiores do que as tabelas ACTION e GOTO para o SLR, o que o torna o método preferido na maioria das situações.

Após apresentar esses dois métodos, concluímos a seção com uma discussão de como compactar as tabelas LR para ambientes com memória limitada.

4.7.1 ITENS LR(1) CANÔNICOS

Apresentamos nesta seção a técnica mais genérica para construir uma tabela de análise LR a partir de uma gramática. Lembre-se de que, no método SLR, o estado i faz uma redução segundo a produção $A \rightarrow \alpha$ se no conjunto de itens I_i tiver o item $[A \rightarrow \alpha]$ e a estiver em FOLLOW(A). Em algumas situações, porém, quando o estado i aparece no topo da pilha, o prefixo viável βa na pilha é tal que βA não pode ser seguido por a em nenhuma forma sentencial à direita. Assim, a redução por $A \rightarrow \alpha$ deverá ser inválida na entrada a .

EXEMPLO 4.51: Vamos reconsiderar o Exemplo 4.48, onde, no estado 2, tínhamos o item $R \rightarrow L$, que poderia corresponder à $A \rightarrow \alpha$ anterior, e a poderia ser o sinal = que está em FOLLOW(R). Assim, o analisador sintático SLR faz uma redução segundo a produção $R \rightarrow L$ no estado 2 com = como próxima entrada. A ação transferir também é exigida, devido ao item $S \rightarrow L = R$ no estado 2. Contudo, não existe uma forma sentencial à direita na gramática do Exemplo 4.48 que começa com $R = \dots$. Assim, o estado 2, correspondente somente ao prefixo viável L , não deve efetuar a redução de L para R .

É possível incorporar mais informações no estado para nos auxiliar na remoção de algumas dessas reduções inválidas por $A \rightarrow \alpha$. Dividindo os estados quando necessário, podemos fazer com que cada estado de um analisador sintático LR indique exatamente quais símbolos podem seguir um handle α , para o qual existe uma redução possível para A .

A informação extra é incorporada ao estado redefinindo-se os itens para incluir um símbolo terminal como um segundo componente. A forma geral de um item agora é um par $[A \rightarrow \alpha : \beta, a]$, onde o primeiro componente $A \rightarrow \alpha\beta$ é uma produção, e a é um terminal ou o marcador do fim da entrada, \$. Chamamos esse objeto de um *item LR(1)*. O 1 refere-se ao tamanho do segundo componente, denominado lookahead do item.⁹ O lookahead não tem efeito algum em um item da forma $[A \rightarrow \alpha\beta, a]$, onde β não é ϵ , mas um item na forma $[A \rightarrow \alpha \cdot a]$ requer uma redução segundo a produção $A \rightarrow \alpha$ somente se o próximo símbolo da entrada for a . Assim, somos forçados a reduzir segundo $A \rightarrow \alpha$ somente sob aqueles símbolos de entrada a para os

⁹ Lookaheads que são cadeias de tamanho maior que um são possíveis, naturalmente, mas não consideraremos esses lookaheads neste texto.

Figura 3 — Trecho do livro sobre itens LR(1) e uso de lookahead para reduzir conflitos. Fonte: Aho, Sethi e Ullman, Cap. 4.

10. Impressão textual da AST

Uma impressão textual determinística facilita testes, depuração e comparação entre implementações. O formato deve indicar o tipo do nó, seus atributos essenciais e a hierarquia dos filhos.

```
Program
  VarDecl type=int name=x
  Assign
    Identifier name=x
    Binary op=*
      Literal value=2
    Binary op=+
      Literal value=3
      Literal value=4
```

Há duas decisões importantes. A primeira é escolher uma ordem fixa para os campos e filhos. A segunda é decidir quais detalhes não devem aparecer. Se a saída incluir endereços de memória, objetos internos ou formatação instável, os testes se tornarão frágeis.

Uma representação em prefixo, como `Binary(*, Literal(2), Binary(+, Literal(3), Literal(4)))`, é compacta. Uma representação indentada é mais confortável para leitura humana. O projeto pode manter as duas: uma saída de depuração para o estudante e uma forma canônica para os testes automatizados.

Regras recomendadas

- um nó por linha;
- dois espaços por nível de indentação;
- atributos escalares em ordem fixa;
- listas vazias impressas explicitamente quando forem relevantes;
- lexemas entre aspas e caracteres especiais escapados;
- nenhum detalhe dependente da execução, como endereço de ponteiro.

11. Testes de integração

O teste de integração verifica o fluxo completo: texto, scanner, parser, AST e diagnóstico. Não basta testar o parser com tokens fabricados se o objetivo é garantir que o scanner entrega exatamente os tipos, lexemas e posições esperados.

Grupo	Exemplos	Verificação
Válidos mínimos	<code>int x; x = 1;</code>	tokens, AST e aceitação
Precedência	<code>x = a + b * c;</code>	hierarquia correta dos operadores
Controle	<code>if (x) y = 1; else y = 2;</code>	ramos e associação do else
Funções	<code>x = f(a, 2);</code>	callee e argumentos ordenados
Inválidos	<code>x = ; / int ; / f(1,;</code>	erros, posições e recuperação

Uma suíte inicial deve conter casos pequenos, pois eles isolam a causa da falha. Depois, devem ser adicionados casos de fronteira: programa vazio, bloco vazio, chamada sem argumentos, expressão entre parênteses, múltiplos erros na mesma entrada e tokens no fim inesperado.

Critério de aprovação

- o scanner produz a sequência esperada de tokens;
- o parser aceita entradas válidas e rejeita inválidas;
- a AST textual é estável e respeita a precedência;
- cada erro informa posição e contexto;
- a recuperação não corrompe os diagnósticos seguintes.

12. Atividade prática — Etapa 2 do projeto

Implemente ou documente a integração do scanner com o parser da MINIC. O resultado deve permitir observar, para cada entrada, os tokens reconhecidos, a decisão sintática, a AST produzida e os erros encontrados.

Parte A — contrato

- documente a estrutura Token e os tipos reconhecidos;
- defina como linha, coluna e lexema são transportados;
- estabeleça o marcador de fim de entrada.

Parte B — AST

- defina nós para programa, declaração, função, bloco, expressão, atribuição e chamada;
- construa a AST durante as ações do parser;
- imprima a árvore em formato textual canônico.

Parte C — testes

- teste um programa com declaração e atribuição;
- teste precedência em $a + b * c$;
- teste if-else, while, retorno e chamada de função;
- teste pelo menos três entradas inválidas e registre as mensagens esperadas.

Exercício 2.3.2: Construa um esquema de tradução dirigido por sintaxe que traduza expressões aritméticas da notação pós-fixada para a notação infixada. Dê árvores de análise anotadas para as entradas $95-2^*$ e 952^*- .

Exercício 2.3.3: Construa um esquema de tradução dirigido por sintaxe que traduza de números inteiros para romanos.

Exercício 2.3.4: Construa um esquema de tradução dirigido por sintaxe que traduza de números romanos para inteiros.

Exercício 2.3.5: Construa um esquema de tradução dirigido por sintaxe que traduza expressões aritméticas pós-fixadas para expressões aritméticas infixadas equivalentes.

2.4 ANÁLISE SINTÁTICA

A análise sintática é o processo para determinar como uma cadeia de terminais pode ser gerada por uma gramática. Na discussão desse problema, é conveniente pensarmos em uma árvore de derivação sendo construída, embora, na prática, um compilador não precise construí-la. No entanto, um analisador sintático precisa, em princípio, ser capaz de construir a árvore, do contrário a tradução não terá garantias de ser correta.

Esta seção introduz um método de análise sintática chamado “descida recursiva”, que pode ser usado tanto para fazer a análise quanto para implementar tradutores dirigidos por sintaxe. A seção seguinte mostra um programa Java completo, implementando o esquema de tradução da Figura 2.15. Uma alternativa viável é usar uma ferramenta de software para gerar um tradutor diretamente de um esquema de tradução. A Seção 4.9 descreve essa ferramenta, Yacc, que pode implementar o esquema de tradução da Figura 2.15 sem modificação.

Para qualquer gramática livre de contexto, existe um analisador que gasta no máximo $O(n^3)$ para analisar uma cadeia de n terminais. Mas o tempo cúbico geralmente é muito dispendioso. Felizmente, para as linguagens de programação reais, em geral podemos projetar uma gramática que pode ser analisada rapidamente. Os algoritmos de tempo linear são suficientes para analisar basicamente todas as linguagens usadas na prática. Os reconhecedores sintáticos de linguagem de programação quase sempre fazem uma única leitura da esquerda para a direita sobre a entrada, examinando à frente um símbolo terminal de cada vez, e construindo partes da árvore de derivação enquanto prosseguem.

As maiores partes dos métodos de análise sintática estão em uma de duas classes, chamadas métodos *descendentes* e *ascendentes*. Esses termos se referem à ordem em que os nós na árvore de derivação são construídos. Nos analisadores descendentes, a árvore é construída de cima para baixo, ou seja, da raiz para as folhas. A popularidade dos analisadores descendentes se deve ao fato de que analisadores eficientes podem ser construídos mais facilmente à mão usando esses métodos. Contudo, a análise ascendente pode tratar de uma classe maior de gramáticas e esquemas de tradução, de modo que as ferramentas de software para gerar analisadores diretamente a partir de gramáticas normalmente utilizam métodos ascendentes.

2.4.1 ANÁLISE SINTÁTICA DESCENDENTE

Introduzimos a análise descendente apresentando uma gramática que é bem simples, além de adequada, para essa classe de métodos. Mais adiante nesta seção, consideramos a construção de analisadores sintáticos descendentes em geral. A gramática na Figura 2.16 gera um subconjunto dos comandos de C ou Java. Usamos os terminais em negrito **if** e **for** para as palavras-chave “if” e “for”, respectivamente, a fim de enfatizar que essas seqüências de caracteres são tratadas como unidades, ou seja, como símbolos terminais. Além do mais, o terminal **expr** representa expressões; uma gramática mais elaborada usaria um não-terminal *expr* e teria produções para esse não-terminal. Da mesma forma, **other** é um terminal representando outros comandos.

```

stmt      →  expr ;
           |  if (expr) stmt
           |  for (optexpr ; optexpr ; optexpr ) stmt
           |  other

optexpr   →  ε
           |  expr

```

FIGURA 2.16 Uma gramática para alguns comandos em C e Java.

A construção descendente de uma árvore de derivação como a da Figura 2.17 é feita iniciando-se na raiz, rotulada com o não-terminal inicial *stmt*, e realizando repetidamente os dois passos a seguir.

1. No nó N , rotulado com o não-terminal A , selecione uma das produções para A e construa filhos em N para os símbolos no corpo da produção.

Figura 4 — Integração do analisador léxico e do analisador sintático na arquitetura do compilador. Fonte: Aho, Sethi e Ullman, Cap. 1.

13. Síntese e checklist

A integração scanner–parser–AST transforma a análise sintática em uma etapa produtiva do compilador. O scanner identifica unidades; o parser organiza relações segundo a gramática; as ações semânticas criam nós; e os testes verificam o contrato entre todas essas partes.

Ideias centrais

- a AST não é uma cópia dos tokens nem uma árvore de derivação completa;
- o momento de criar um nó depende da estratégia de parsing, mas os dados necessários devem estar disponíveis;
- declarações, expressões, comandos e chamadas precisam de estruturas distintas;
- posições de linha e coluna devem acompanhar tokens e nós;
- mensagens de erro devem orientar a correção e permitir recuperação controlada;
- a impressão textual canônica torna a AST observável e testável.

Checklist de aprendizagem

Pergunta	Consigo responder?
Quais campos um token precisa fornecer ao parser?	tipo, lexema, linha, coluna e fim de entrada.
Como uma redução pode criar um nó?	remove os filhos do lado direito e empilha o nó do lado esquerdo.
Por que a AST omite E, T e F?	porque esses não terminais são auxiliares da gramática, não operações da linguagem.
Como testar o front-end?	com entradas válidas, inválidas, casos de fronteira e comparação da AST textual.

Leitura orientada

AHO, A. V.; SETHI, R.; ULLMAN, J. D. *Compiladores: princípios, técnicas e ferramentas*. Capítulo 2, Seções 2.4 e 2.5; Capítulo 4, Seções 4.7 a 4.9. Material fornecido pelo docente.

Próxima conexão: a AST construída nesta aula será a entrada da análise semântica, da tabela de símbolos e da verificação de tipos.